Reflection on Advanced Topics

Names: Atal Khogyanai & Ruben **Student number:** 2485818 & …

Project: Tic Tac Toe (Java Swing) **Course:** Java Programming (2IP90)

Topic 1: Usage of GitHub

Why

I chose this topic because it seemed like a useful skill to learn. Many developers, including game developers, use Git and GitHub, and I figured there had to be a reason for that: either it makes coding easier, or it is simply convenient to use. It did turn out to be a very helpful tool. I could see every update and change, and publish my changes online. That said, things got chaotic at times when my partner and I worked on different things at the same time.

What I learned

The main thing I learned was how to use Git, through Git Bash, and GitHub. I had never used either before, so it took some getting used to. Typing commands in Git Bash to commit and push my files, merging changes on GitHub, and connecting it all to Apache NetBeans were all new to me, but over time they became routine.

How I applied it

In my project I used Git by creating a separate branch for every feature, such as player moves, turn-timer, main-menu, minimax, arcade, ui improvements etc . I did this because it would be very chaotic and not sorted properly if I handled everything in one main file. Each time a feature was finished, I committed it in Git Bash, pushed it, merged the pull requests on Github and pulled it on Git Bash. This meant that each part was handled separately, and if there was ever a broken feature, we could specifically see what part was invalid.

What I learned from the implementation

The biggest challenge was working in a sorted manner. There were cases where we worked on the same thing at the same time, and also pushed changes in the repo at the same time. This caused our files to sort of ‘’merge’’ together and break. This happened because [your guess at the cause]. Later in the project I made another mistake, which was that I forgot to push my changes made to the UI, and eventually losing all the progress. Later I found out I didn’t actually lose the progress, but it turned out that I could just push it again. From this I learned that Git is safer than it feels, because almost nothing gets deleted by accident. If I used Git in a team again, I would agree beforehand who works on which feature and always use separate branches.

Topic 2: User Experience

Why

After my tutor saw the first version, she said it didn't look polished. When I looked at my game again, I noticed that it did look very dull and basic, and lacked reactivity/user friendliness. I chose user experience because if a game isn't user friendly or clean, no one would want to play it or would enjoy playing it.

What I learned

While researching I found Nielsen's usability heuristics, a set of principles for designing interfaces that are easy to understand and use, so that people can use a program without needing someone to explain it. The ones most relevant to my game were visibility of system status (who wins), error prevention (not being able to click the same square), and a help tab. What surprised me was that, I had initially thought UX was mainly design, but it's also reactivity and explaining the game in a friendly way. For example, a ''help'' tab, or hovering over a square and seeing possible options, or explaining power-ups in arcade mode.

How I applied it

I linked each improvement to one of these principles. For visibility of system status, I added the turn label to be in the active player's color, and also the timer shown in the top right.

For error prevention, like I said, not being able to click the same square, and ''bomb mode'' powerup only being available on valid squares.

For user control, I added a play again and menu button.

There's alot more, such as having one theme class for all colors and fonts (consistency), and a help screen (help and documentation).

What I learned from the implementation

The hardest part of improving the user experience was the hovering over a square feature. The real challenge was that first, a square didn't know whose turn it is. I solved this by making the class GamePanel update every square after each move. I also had to make choices about what not to add. For example, I first wanted to add a timer warning, telling you when your timer is low. I ended up not doing this because I felt like it was unnecessary and would make the game look clunky. The timer is only 10 seconds so if you see a red warning every turn it's just annoying. Overall, the changes made the game feel way more lively and enjoyable, and I learned that user friendliness is incredibly important in any good game.